Coursework #2 – Group Assignment (30%)

Module's Information:

Module	SDT4202 SOFTWARE QUALITY	
Session	JUNE 2026	
Programme	BCSI	
Lecturer	**NOOR ZUHAILI MD YASIN**	
	Email: noorzuhaili.mdyasin@newinti.edu.my	Room: A3A - 18
Coursework Type	GROUP ASSIGNMENT	
Percentage	30%	
Hand-out Date	**WEEK 1**	
Due Date	**WEEK 4 (25/6/2026)**	

Student's Declaration:

I declare that:

- **I understand what is meant by plagiarism**
- **This assignment is all my own work and I have acknowledged any use of the published or unpublished works of other people.**
- **I hold a copy of this assignment which I can produce if the original is lost or damaged**

No.	Name & Student ID	Section	Total Marks	Signature
1.	SHAHADAT HOSSAIN	6MB1		SHAHADAT
2.	MD ARIF MAHMUD OYION	6MB1		ARIF
3.	KEENJHAR LASHARI	6MB1		KEENJHAR
4.	KOONJ	6MB1		KOONJ

DATE: 25/06/2026

Learning Outcomes Assessed:

LO2	Evaluate the likely outcomes of planned and unplanned change on a workforce, particularly changes in Information Systems organization.

Rubric

A. Group Assessment-Report					
Element(s)/ Marks	1-3	4-5	7-8	9-10	Marks
Introduction	The points are not clear and not accurate, but the introduction is at least related to the topic with no justification	The points are less clear and less accurate with few justification	The points are clear and precise but less accurate with comprehensive justification	The points are clear and accurate with comprehensive justification	
Element(s)/ Marks	1-5	6-10	11-15	16-20	Marks
Change management	The points are not clear and not accurate, but the findings are at least related to the topic with no design/example /Justification	The points are less clear and less accurate with few design/example/ justification	The points are clear and precise but less accurate with comprehensive design/example/ justification	The points are clear and accurate with comprehensive design	
Version management	The points are not clear and not accurate, but the findings are at least related to the topic with no design/example /Justification	The points are less clear and less accurate with few design/example/ justification	The points are clear and precise but less accurate with comprehensive design/example/ justification	The points are clear and accurate with comprehensive design	
System building	The points are not clear and not accurate, but the findings are at least related to the topic with No design/example/ justification	The point of the partner is less clear and less accurate with few design/example/ justification	The points are clear and precise but less accurate with comprehensive design/example/ justification	The points are clear and accurate with comprehensive design	
Release management	The points are not clear and not accurate, but the findings are at least related to the topic with no design/example/ justification	The point of the partner is less clear and less accurate with few design/example/ justification	The points are clear and precise but less %accurate with comprehensive design/example/ justification	The points are clear and accurate with comprehensive design	
Element(s)/ Marks	1-3	4-5	7-8	9-10	Marks
Format	The font is not consistent, diagram is not clear, no table of content, no page number, no student id	Most of the font is less consistent, diagram is less clear, no table of content, no page number, no student id	Some of the font is less consistent, some diagram is less clear, with table of content, page number, student id	Most of the font is consistent, diagram is clear with label, with table of content, page number, student id	

Change Request Form

[This form is divided into three sections. Section 1 is intended for use by the individual submitting the change request. Section 2 is intended for use by the Project Manager to document/communicate their initial impact analysis of the requested change. Section 3 is intended for use by the Change Control Board (CCB) to document their final decision regarding the requested change.]

Project Name	Facebook Platform Security Enhancement Project	Date	15/03/2024
Project Number	META-FB-2024-SEC-001	Requestor	Meta Security Engineering Team
Project Manager	Facebook Security Project Manager	Project Owner	Meta Platforms, Inc.

1.) SUBMITTER - GENERAL INFORMATION			
Change Request			
Type of CR	Enhancement ☐	☑ Defect	
Project/Program/Initiative	Facebook Password Reset Security Improvement Initiative		
Submitter Name	Meta Security Engineering Team		
Brief Description of Request	Modify the Facebook password reset mechanism to eliminate the account takeover vulnerability discovered in 2024.		
Date Submitted	15/03/2024		
Date Required	25/03/2024		
Priority	Low ☐	Medium ☐	High ☐ Mandatory ☑
Reason for Change	A weakness in the password reset process could allow unauthorized individuals to gain access to Facebook user accounts.		
Other Artifacts Impacted	Password Reset Module,Security Requirements Documentation Test Cases,User Support Documentation		
Assumptions and Notes	The revised password reset mechanism will support existing authentication infrastructure without disrupting normal user activities.		
Comments	The fix should be included in the next emergency security release.		
Attachments or References	Yes ☑	No ☐	
	Link: Internal Security Assessment Report and Vulnerability Analysis		
Approval Signature	Meta Security Engineering Lead	**Date Signed**	15/03/2024

Details:

Describe the Requested Change
Update the password reset process by implementing stronger identity verification mechanisms, enhanced validation procedures and additional authentication controls to prevent unauthorized account access.

Risk Identification/Analysis
Delays in deployment may expose users to further attacks.
New authentication controls may affect user convenience.
Extensive regression testing is required.

Impact Analysis

Work Products to be Modified	Version Number
1.Password Reset Module	v4.3
2.Security Requirements Specification	v2.5
3.Security Test Cases	v3.1

Existing password recovery functionality, testing procedures and related documentation must be updated and revalidated to ensure compatibility with the enhanced security controls.

Quality Impact

Additional Quality Assurance or Quality Control Activities

1.Security Testing

2.Penetration Testing

3.Regression Testing

The enhancement requires extensive security validation to ensure that the vulnerability has been eliminated without introducing new defects.

Schedule Impact

New Deliverables Description	Effort Hours	Date Required	Impact to Other Delivery Dates
1.Password Reset Security Enhancement	40 hrs	25/03/2024	Minor
2.Security Testing Report	24 hrs	27/03/2024	Minor
3.Updated Security Documentation	8 hrs	28/03/2024	None

Estimated implementation date: 25/03/2024.
New estimated project completion date: 30/03/2024.

Budget Impact

New Deliverables Description	Lessen or Eliminate Other Expenses? Please describe.	Cost of New Deliverable	Total
1.SecurityEnhancement Development	No	$8,000	$13,000
2.Security Testing Activities	No	$4,000	
3.Documentation Updates	No	$1,000	

The change will increase the project budget due to additional development, security testing and documentation updates. However, the extra cost is necessary to improve account security and prevent future financial and reputational losses.

2.) PROJECT MANAGER - INITIAL ANALYSIS

Hour Impact	72 Hours	[Enter the hour impact of the requested change]	
Duration Impact	5 Days	[Enter the duration impact of the requested change]	
Schedule Impact	WBS 2.1,WBS 3.2,WBS 4.1	[Detail the impact this change may have on schedules]	
Cost Impact	$13,000	[Detail the impact this change may have on cost]	
Comments	Immediate implementation is recommended because the vulnerability directly affects user trust and platform security.		
Recommendations	Approve and prioritize the change as an emergency security fix.		
Approval Signature	Facebook Security Project Manager	**Date Signed**	15/03/2024

Decision			
Approved	☑	Rejected	☐
Approved with modifications	☐	Deferred	☐
Justifications			
The password reset vulnerability poses a significant security risk that may lead to unauthorized account access. Implementing the enhancement will strengthen user protection, reduce security threats, and maintain Meta's reputation. The benefits outweigh the additional cost and schedule impact.			
Additional Comments			
The implementation should be monitored closely, and all security validation activities must be completed before deployment. User experience should also be evaluated to ensure minimal disruption.			

3.) CHANGE CONTROL BOARD – DECISION			
Decision	Justifications ☑	Rejected ☐	More Info ☐
Decision Date	16/03/2024		
Decision Explanation	The vulnerability presents a high risk to user accounts and Meta's reputation. The benefits of immediate remediation outweigh the associated costs and schedule impacts.		
Conditions	The updated password reset mechanism must successfully pass penetration testing and regression testing before public deployment.		
Approval Signature	Meta Change Control Board Chairperson	**Date Signed**	16/03/2024

Meta Change Control Board Chairperson Name 16/03/2024

Approver's Printed Name Date

Meta Change Control Board Chairperson Meta Change Control Board Chairperson's Signature

Title Signature

☐

TABLE OF CONTENTS

SERIAL	TOPICS	PAGE NUMBER
1.	INTRODUCTION	9
1.1	BACKGROUND OF THE ORGANIZATION	9
1.2	THE SOFTWARE SYSTEM USED	9-10
1.3	OBJECTIVE	10
1.4	PURPOSE OF THIS REPORT	11
1.5	PROBLEM OVERVIEW	11-13
2.	CHANGE MANAGEMENT	13
2.1	INTRODUCTION TO CHANGE MANAGEMENT	13-14
2.2	TYPES SOFTWARE CHANGES	14-15
2.3	IMPORTANCE OF SOFTWARE QUALITY CHANGE MANAGEMENT	15
2.4	SOFTWARE QUALITY PROBLEM RELATED TO CHANGE MANAGEMENT IN META	16-24
2.4.1	ISSUE 1 CHICAGO, ILLINOIS, 2018 : CAMBRIDGE ANALYTICA DATA SECURITY BREACH	16-18
2.4.2	ISSUE 2 : 533 MILLION USER DATA SCRAPING VULNERABILITY (2019–2021)	18-20
2.4.3	ISSUE 3 GLOBAL PLATFORM OUTAGE DUE TO FAULTY CONFIGURATION CHANGE(OCTOBER 2021)	20-22
2.4.4	ISSUE 4 ACCOUNT TAKEOVER VULNERABILITY IN PASSWORD RESET SYSTEM (2024)	22-23
2.4.5	Good Practices of Change Management for META	23-24
2.5	SUMMARY OF CHANGE MANAGEMENT ISSUES AND SOLUTIONS	25
2.6	THE MODEL DEVELOPED FOR CHANGE MANAGEMENT FOR META	25-26
3.	VERSION MANAGEMENT	26-32
3.1	VERSION MANAGEMENT INTRODUCTION	26-27
3.2	VERSION MANAGEMENT'S OBJECTIVE	27
3.4	META'S SOFTWARE QUALITY ISSUES EMERGING DUE TO POOR VERSION MANAGEMENT	28-29
3.4.1	ISSUE 1: CAMBRIDGE ANALYTICA DATA BREACH (2018)	28

3.4.2	ISSUE 2: Incident No. 34219 User Data Scraping Incident.	28
3.4.3	ISSUE 3: GLOBAL PLATFORM OUTAGE (2021)	29
3.4.4	ISSUE 4 FAILURE OF VERSION MANAGEMENT	29
3.5	META'S BEST PRACTICES FOR VERSION MANAGEMENT	30-31
3.6	META RECOMMENDED VERSION MANAGEMENT MODEL	31-32
3.7	CONCLUSION	32
4.	SYSTEM BUILDING BY PROVIDING THE INTERFACE(S) OF THE SYSTEM BUILD	
4.1	OVERVIEW OF THE SYSTEM	33
4.2	SYSTEM ARCHITECTURE OVERVIEW	33-34
4.3	INTERFACE DESIGN	
4.3.1	LOGIN PAGE	35
4.3.2	REGISTRATION PAGE	36
4.3.3	HOME FEED PAGE	37
4.3.4	USER PROFILE PAGE	38
4.3.5	FRIEND REQUEST PAGE	39
4.3.6	MESSAGING PAGE	40
4.3.7	NOTIFICATION PAGE	41
4.3.8	SETTINGS PAGE	42
4.4	SOFTWARE QUALITY IMPROVEMENTS ACHIEVED THROUGH THE DESIGN	43
4.5	SYSTEM WORKFLOW SUMMARY	43
5.	RELEASE MANAGEMENT	43-50
5.1	PROBLEMS IDENTIFIED	44
5.2	COMPANY RELEASE AND THE IMPORTANCE OF RELEASE	44-45
5.3	PROJECT HAIR COLOURATION RECOVERY PLAN	45-47
5.3.1	PLANNING FOR RELEASE	45-46
5.3.2	BUILD AND PRE-RELEASE VALIDATION	46
5.3.3	RELEASE APPROVAL PROCESS	46-47

5.3.4	CONTROLLED DEPLOYMENT	47
5.4	PROPOSED RELEASE WORKFLOW	48
5.5	ROLLBACK AND RECOVERY PLAN	48-49
5.6	RECOMMENDATION FOR THE IMPROVEMENT OF RELEASE MANAGEMENT FOR THE IDENTIFIED ISSUES	49
5.7	SAMPLE FOR A DIGITAL CONNECT SOLUTIONS RELEASE PLAN	49-50
5.8	THE BENEFITS OF A GOOD RELEASE MANAGEMENT PROCESS	50
5.9	RELEASE MANAGEMENT CONCLUSION	50
6.	CONCLUSION	51-53
6.1	THIS REPORT DEALS WITH FOUR ISSUES	51
6.2	A SUMMARY OF THE KEY FINDINGS	51-52
6.3	KEY RECOMMENDATIONS	52
6.4	CLOSING STATEMENT	53
7.	REFERENCES	53-56
8.	APPENDIX	57-59

1. INTRODUCTION

1.1 BACKGROUND OF THE ORGANIZATION

Meta or Meta Platforms, Inc., is one of the world's biggest technology firms. The company was established in 2004 as a Facebook, Inc. before being renamed to Facebook, Inc. in 2005. In October 2021, the company's name was officially changed to Meta Platforms, Inc. due to the expansion of its business outside of social media. Meta currently has a suite of the most popular digital platforms globally, such as Facebook, Instagram, WhatsApp, Messenger and Threads.

Meta has its headquarters located in Menlo Park, California, USA. The company is a cloud-based technology company that provides more than 3 billion active users daily through its applications. Meta's primary source of revenue is digital ads. Meta's platforms are a great way for businesses and brands to advertise to their target audience from around the world.

Meta's main software system is a social media and communication platform that operates in the cloud. Users can create individual user accounts, share content, communicate with friends & family, make payments, join groups, watch videos and interact with businesses on this platform. A large number of new users are added to the Meta system every day.

Meta's software systems are very large and complex due to their global scope and size. Even a small software problem can affect millions or billions of users at the same time. This places software quality as one of the most vital issues that Meta has to look into as a business.

1.2 THE SOFTWARE SYSTEM USED:

MATLAB, Microsoft Office.

Meta's Facebook web and mobile application (Facebook app) has been chosen as the software system for this report. This can be considered as a cloud platform and mobile app which are both suitable for this assignment.

Using the Facebook platform gives people the power to:

- Set up and maintain own accounts
- Explore and post in their News Feed

- Using messenger, send and receive messages
- Make payments using Facebook Pay
- Connect and engage with groups and communities
- Recording activities and connections with businesses
- Communicate with third party applications via Facebook's API

Meta's private cloud infrastructure consists of millions of machines in data centers around the world, including in the United States, Europe and Asia, which run the Facebook platform itself. About 2.9 billion people are active monthly users of the platform. This scale has the consequence that Facebook is one of the most demanding software systems in the world with regard to performance, reliability, security and maintainability.

1.3 OBJECTIVE

This report aims to identify and analyse three to four real software quality issue(s) that Meta's Facebook platform has faced from the year of 2018 onward. This report will also discuss, for each detected problem, how Meta has addressed that problem, as well as presenting improvement options that make use of the correct Software Quality Assurance (SQA) practices.

Additionally, this report highlights some key prerequisite areas of software engineering pertinent to the problems discovered. These concepts include:

- **Change Management:** How changes of software have to be controlled and managed properly
- **Version Management:** The management of different versions of the software , how they should be tracked and maintained.
- **System Building**: The process of software building and its deployment.
- **Release Management:** Safely releasing new versions of the software to users

Finally, depending on the characteristics of the encountered issues and the platform requirements, a proper SDLC model is proposed for Meta at the end of this report.

1.4 PURPOSE OF THIS REPORT

This report is specifically on the Facebook platform and the software quality problems Meta experienced during 2018 - 2024. The problems detailed in this report are actually events that are documented in public and confirmed by Meta, regulators and reputable tech news outlets.

This report discusses four problems with software quality:

Issue 1: Cambridge Analytica Data Privacy and Security Breach: (2018)

Issue 2: A vulnerability was uncovered in scraping 533 million user data.

Issue 3: A faulty configuration change has resulted in an outage across all the Global Platform (October 2021).

Issue 4: This vulnerability is known as a Facebook Account Takeover in the password reset system for 2024.

Quality measures such as the ISO 9126 quality factors, the McCall's Quality Factor model and the IEEE Standard definition of software defects, are used to analyse each issue.

1.5 PROBLEM OVERVIEW

Software Quality Assurance (SQA) is a process of planned and systematic activities to provide reasonable assurance that an item of product will meet the technical requirements. It is also a series of activities for testing the process used in the product development or production.

Even though Facebook software is among the most upscale software in the world, it has encountered severe software quality failures. The resulting failures had significant ramifications not only for the software itself, but for billions of users, the company's business results and their reputation around the world.

The issues that Meta is grappling with are very like those mentioned in the assignment background scenario. Meta added that they had just like DigitalConnect Solutions in the case study, the following:

- Security weaknesses which allowed access to private information of users by unauthorized people
- Cases when system failures were affecting the full availability of the platform to the users
- Reliability issues where the system could not maintain its performance level under normal conditions
- Bad configuration management due to one misconfiguration, the whole world was offline for billions of people.

The problems highlight that even a large scale and well invested company like Meta can face poor software quality practices. This report will examine each problem in detail and provide suitable SQA-based solutions and improvement plans.For META to launch a successful competition, the significance of software quality must be taken into account.

There are two important points of view from which to consider quality:

- **Conformance to requirements (Crosby, 1979) :** Anything not conforming to the requirements is called a defect. A good-quality product has fewer faults or deficiencies. The requirements must be clearly stated and the product must conform to them.
- **Fitness for use (Juran, 1970) :** A good quality product fulfills the need of the user and will give better satisfaction to the user.

Both these viewpoints are crucial to Meta. There are strict functional, performance and security requirements that the Facebook platform has to meet. Meanwhile, it has to be able to meet everyone's demands, as billions of users rely on users around the globe for information, business and communication on a daily basis.

Meta will have repercussions more than technical when their software quality suffers. They include:

- Billions of users losing access to the platform
- Millions of users' sensitive personal information are made public
- Billions of dollar fines from regulators
- Loss of trust in the company and users

- Major business downturn and loss of confidence in advertisers

Exactly like the domino effect explained in the assignment background one software quality failure cascades to the rest of the organisation. This report illustrates the effectiveness of proper SQA practices, change management, version management, system building and release management in preventing or limiting the effects from these issues and how Meta has struggled over the years to raise its software quality standards.

2. CHANGE MANAGEMENT

2.1 INTRODUCTION TO CHANGE MANAGEMENT

Change management is among the key components of software quality assurance. Change is inherent in the Software Development Life Cycle. Software systems are continually updated, enhanced, corrected and extended. New features are added, old bug fixes restored, security holes plugged and system configurations changed on a regular basis. Even if the change is small and simple, it can seriously affect the entire software system if a proper and structured process is not in place to handle these changes.

Change management may be defined as a structured discipline that handles controlling and handling of any kind of changes made to the software program system all through its life duration. The key aspect of change management is to ensure that all changes to the software are appropriately planned, reviewed, approved, tested and documented prior to applying them to the live system. Doing so allows to avoid the introduction of unforeseen defects in the system due to uncontrolled or poorly-managed changes.

Change Management in the context of Software Quality Assurance is closely related to the target of assuring an acceptable degree of confidence that the software will meet the functional technical requirements. If software is not subjected to proper control when being changed, the resulting software may not meet its requirements, thus resulting in the introduction of defects and the degradation of the software quality. That's precisely what occurred in a number of Meta's big software quality mishaps.

The issue of software faults is also near to the idea of change management. There are nine known causes of software defects, several of which are directly related to poor change management practices. The latter includes instances of poorly-defined requirements,

intentional deviations from the software conditions, programming mistakes and failure to follow instructions in software documentation. With a robust and organised change management process all these causes can be avoided or minimised.

Change management has grown significantly in importance and for businesses such as Meta, with one of the world's largest and most complex software platforms, it's absolutely essential. Meta engineers make thousands of code changes daily. Any of those changes could potentially cause a break in the entire platform, operationally impacting billions of users simultaneously, if not sensibly managed.

2.2 TYPES SOFTWARE CHANGES

To get a sense of how change management is done at Meta, it is crucial to appreciate past what kinds of changes occur in a software system. During the maintenance phase of a software system, changes follow some of the main types that are based on the software life cycle as follows:

Corrective Maintenance: Changes made to the software to fix faults or defects found in that software are called Corrective Maintenance. These are “reactive” changes – they occur after an issue has been identified. For instance, in 2024, when Meta was informed about the brute-force attack vulnerability that was discovered in its password reset feature, the patch that Meta provided is a corrective maintenance change.

Adaptive Maintenance: Modification of a software to fit a new or altered environment. For instance, with new operating systems or new regulations mandating different data storage and processing methods, Meta needs to make some adaptive changes to its platform in order to keep it functioning properly in the new environment.

Perfective Maintenance: Where a change is required to improve the software by adding new functionality or updating old functionality. It's the most frequently seen sort of shift in Meta. Changes to the Facebook platform like Stories, Reels, Marketplace and more – are all perfective maintenance changes.

Preventative Maintenance: It includes changes that can prevent problems that may happen later. For instance, following the worldwide outage in 2021, Meta changed how it manages its network configurations to avoid a similar incident in the future. They are about making proactive changes to enhance long term sustainability and reliability of the system.

It is essential to understand these types of changes as the significance of each type of change varies and requires many types of reviews and approvals before applying to the live environment.

2.3 IMPORTANCE OF SOFTWARE QUALITY CHANGE MANAGEMENT

Change management has a significant part in software quality maintenance. According to ISO 9126, maintainability is a set of attributes that address the effort required to make specified changes to software, as well as to identify the causes of software failures, correct the failures and validate the corrections. As seen in this definition, maintainability, one of the six critical factors of quality, can be directly related to the manner in which changes are handled within the software system.

When change management is done properly, it brings the following benefits to a software system:

- All changes are duly vetted and tested against before they are rolled out and thus the chance of adding fresh faults is reduced.
- Any modifications are correctly recorded hence problems are easily tracked to their origin should anything go wrong.
- These changes are done in a controlled, orderly fashion and thus less opportunity for multiple changes to conflict with one another
- This software is stable and dependable over time since it continues to adapt and develop over time.
- Helping the development team to quickly determine what changed, when it changed and who changed it will improve the speed of getting an issue fixed.

Change management processes can have an impact if they fail, or if they are not implemented. Uncontrolled changes can lead to new defects, system jams and outages,

broken functionality and security flaws. Which is precisely what occurred with Meta a number of times as shall be detailed in later sections.

2.4 SOFTWARE QUALITY PROBLEM RELATED TO CHANGE MANAGEMENT IN META

Meta has suffered a number of significant software quality failures that were directly attributable and/or exacerbated by lack of change system practices. This section analyses each of these issues in detail.

2.4.1 ISSUE 1: CAMBRIDGE ANALYTICA DATA SECURITY BREACH 2018

WHAT HAPPENED:

Cambridge Analytica, a political consulting firm was revealed to have harvested the personal data of around 87 million Facebook users without their knowledge or consent in 2018. Facebook had made a change to its Graph API (programming interface used by third-party applications to access users' Facebook data) which was the root cause of this problem.

The reason behind the development of the Graph API was for Facebook to enable third party app developers to acquire user's data to create helpful apps. But here comes the substantial development in API that enabled developers to access not only all the data of those who have installed their app, but even the data of all their friends without any consent. This API change meant that this style of operation was highly secure and was exploited by a third party quiz app used by Cambridge Analytica.

The issue was that this adjustment to the API hadn't been reviewed or controlled from a security standpoint. The change was desired as a benefit to developers to make better applications, though the security aspects of the change were not thoroughly examined. This can be considered a failure of change management; a major change was provoked to the system without taking into account the risks and not setting up adequate controls to prevent misuse.

Type of Defect:

In the terminology of the IEEE Standard 610.12, this is a problem with all three types of defects:

- **Software Error :** The question of consent controls to allow friend data access to the API was a logical design error in the system.
- **Software Fault :** The API functionality which enabled mass data gathering was a wrong process inside the software program system
- **Software Failure :** The system's outward performance was not what users expected or indicative of privacy rules/laws

ISO 9126 Quality Factor:

This is a failure in **Functionality Security**, the ability of the software to prevent unauthorized access and to withstand deliberate attacks to obtain access to confidential information. A few API changes eliminated vital security measures that would otherwise have been in place to safeguard user data.

McCall's Quality Factor:

In fact this is directly relevant to **Integrity** which addresses the security of the system and denial of access to unauthorised users. There was a correlation with **Correctness** too, because the system was not giving the correct results and expected results in terms of data protection.

Cause of Defect

This problem has to do with two primary reasons for software design flaws:

- **Faulty requirements definition** : the security requirements for API access by third parties were not clearly and fully stipulated.
- **Non-compliance with documentation and coding instructions:** The reason it got rejected: Cause 6 "Non-compliance with documentation and coding instructions" the API change didn't meet Facebook's own privacy guidance.

Impacts on Financial and Business:

The Cambridge Analytical scandal arguably left a massive impact on Meta. In 2019, the United States Federal Commerce Commission (FTC) imposed a $5 billion fine on Meta, (at that time) the largest ever recorded of a technology company in the United States. Meta also settled a further $725 million lawsuit with the affected users in 2022. The leak of that private data resulting from the breach was also a huge blow to the brand and reputation of Meta and necessitated regulatory investigations in several countries as well as a hearing in the United States Congress to call Mark Zuckerberg to testify.

How Meta Responded:

In the wake of the Cambridge Analytica debacle, Meta has made a number of efforts to strengthen its change management and security protocols:

- Limit third-party apps using the Graph API to access users' information and no longer give apps access to information about a user's friends without their consent.
- Implemented a formal App Review process, where all 3rd party Applications were subjected to review and approval prior to gaining access to user data.
- Introduced Data Abuse Bounty Program to "incentivize" external researchers to report data abuses by third party developers
- Increased jobs in security and safety by over 15,000 personnel put in place stronger API change review process with API changes for data access features needing security impact assessment

2.4.2 ISSUE 2 : 533 MILLION USER DATA SCRAPING VULNERABILITY (2019–2021)

WHAT HAPPENED:

From 2018 to 2019, malicious actors abused the Facebook Contact Importer feature. The initially created feature was for permitting the user to search for their ‘friend’ on Facebook using the upload of their phone contacts. But attackers found out they could break this feature and do the exact opposite; put in big lists of phone numbers and the system will spit out the information of the Facebook pages that each number belonged to.

This vulnerability was present due to the change of the Contact Importer feature that introduced this unintended behaviour. There were failed security testing for the change before

it was put into the live system. The attacker accessed the system using automated bots, to enter millions of phone numbers and gather corresponding User Profile information. The vulnerability allowed the collection of the personal data of 533 million users of Facebook in 106 countries, by the time Facebook found the vulnerability and patched it in August 2019. This information was later leaked to a hacking forum in April 2021.

Type of Defect:

- **Software Error :** A logical error in design of Contact Importer feature produced an error that permitted it to be utilized in a way not intended.
- **Software Fault :** Feature had the wrong process in place which enabled the mass data collection via automated queries
- **Software Failure :** The system's external behaviour was not as required by its specification — "The intent was to protect user information but this is the opposite of that."

ISO 9126 Quality Factor:

Failure according to **Functionality** , **Security** , **Fault Tolerance** and **Reliability**. There was no place in this system to detect and prevent the automated scraping attacks, demonstrating a lack of security and fault tolerance.

McCall's Quality Factor:

This corresponds to **Integrity** - The system has not been able to stop the unauthorised mass collecting user data. Also, there is reliability as well. The system didn't realize that it was being victimized and couldn't take steps to defend against this sort of attack.

Cause of Defect:

- **Faulty requirements definition:** Failures were in the security requirements definition, specifically protection against automated mass queries was not included for the Contact Importer feature.
- **Shortcomings of the testing process**: Security testing was not able to discover this vulnerability before the feature was deployed.

Financial and business impacts:

In November 2022, Ireland's Data Protection Commission slapped it with a €265 million (~$276 million) GDPR penalty for the data breach. The incident was also a huge damage to the reputation and led to regulatory investigations throughout all of Europe.

How Meta Responded:

- For the vulnerability in the Contact Importer, fixed in August 2019 by implementing Rate Limiting to prevent of automated mass queries
- Implemented anti-scraping control and detection to deter and prevent automated bot activity
- Made API access more restricted to avoid similar misuse of other features
- Made changes to the security testing process to cover automated security abuse scenarios prior to deployment of security features

2.4.3 ISSUE 3 GLOBAL PLATFORM OUTAGE DUE TO FAULTY CONFIGURATION CHANGE(OCTOBER 2021)

WHAT HAPPENED:

Facebook, Instagram, WhatsApp, Messenger and Oculus all experienced a complete outage for about six hours on the day of October 4, 2021. North America's InterSurveys said it was the “global communications failure” of all time, impacting about 3.5 billion users across the world. A faulty configuration change by Meta's network engineers during routine maintenance procedures caused the issue.

The change the engineers implemented involved the configuration of Facebook's backbone routers , network hardware devices that are responsible for coordinating traffic between Meta's data centres globally. The change in configuration had the effect of taking all Facebook's routing information out of the web. This resulted in the internet's hard-coded locations of the Facebook servers going dark, rendering all of Meta's services unreachable.

Things just got worse because the internet had been knocked offline and they were using the same network for Meta's internal recovery tools, monitoring systems and even for employees to access the business. That meant that the engineers were not able to use their usual tools to diagnose and fix the issue remotely. It took almost 6 hours to manually restore network connections, requiring physical travel to Meta's data centers.

This is a textbook example of the impacts that can occur if the change management process is not adhered to properly. The configuration change was part of a routine maintenance activity, which wasn't tested or validated before being applied to the live production system. There also wasn't a robust rollback plan for when things go wrong allowing for engineers to quickly roll things back. In addition, the recovery system was not adequately isolated from the production system, making it extremely difficult to recover from production system failure.

Type of Defect:

- **Software Error :** wrong result due to a human error in making the change
- **Software Fault :** incorrect data definition within software had been considered as an incorrect configuration
- **Software Failure :** the whole system came to a halt to cease the provision of its services and by definition it is the software problem that failed; this is the most obvious reason why failure occurs

ISO 9126 Quality Factor:

This represents a failure of many quality factors:

- **Reliability:** Recoverability the system was incapable of promptly re-establishing its performance level after a failure
- **Efficiency:** Time behaviour, there was a complete lack of response times in the system.
- **Maintainability:** Analyzability the team were unable to easily determine the source of the issues as their diagnostic tools were also down

McCall's Quality Factor:

Reliability: the system failed to give services entirely. It's linked to **Maintainability**; failure to correct the fault required an enormous amount of effort, due to poor system design and poor change management.

Cause of Defect:

Cause 1: Deliberate changes outside of software requirements , the configuration change was outside of the network configuration requirements

Cause 2: Coding errors in this case configuration errors in the backbone router settings.

Cause 3: Shortcuts in the testing process, the configuration change was not properly tested before being applied

Financial and Business impact:

Advertising sales during the six-hour outage were an estimated $60 million loss for Meta. The outage caused Meta's share price to fall about 5% on the date of the incident, costing the company billions of dollars. The shock was also driving damage to the company's reputation and questioning the resiliency of its change management and disaster recovery practices.

How Meta Responded:

- Implementing new configuration change management processes, meaning that changes to critical network equipment are performed with several different review and approvals processes in place before implementation can occur.
- Introduced improvement to testing and validation of changes that affect the configuration prior to deployment in production systems
- Revamped the recovery system to run on a separate network, which is independent of the main production network
- Adopted the Conveyor deployment system, a deployment tool developed by Meta that allows us to deploy changes across all of our services in a safe and consistent way.
- Enhanced automated rollback functionality to easily roll back changes if they run into issues

2.4.4 ISSUE 4: ACCOUNT TAKEOVER VULNERABILITY IN PASSWORD RESET SYSTEM (2024)

WHAT HAPPENED:

A security researcher was able to find a critical flaw in Facebook's password reset in early 2024. Facebook was sending a six-digit authorization code to other devices being used by said users when they requested to reset their password. For about two hours, however, the code was live and there was no software to prevent brute force attack, that is, where a would-be robber could enter the code he or she guessed was correct, but if wrong it simply rang the second bell and tried again. He could have towed about the account by having the login name of the victim user.

This weakness was due to the lack of security testing of the change that enabled this password reset function to be deployed. This modification's security would have been

disastrous, as it did not provide any form of protection against brute force. The vulnerability was reported via the Bug Bounty Program of Meta who then patched it.

Type of Defect:

- **Software Error:** A logical error with the reset function of the software meant it could be accessed using brute force, known as Software Error.
- **Software Fault:** Lack of brute force protecting was incorrect process in the authentication system
- **Software Failure**: The authentication system failed to do its job and stop someone accessing someone's accounts unauthorizedly.

ISO 9126 Quality Factor:

This is an Instance of **Functionality** : Security, Reliability and Fault Tolerance. No protection for repeated attempts to access the system unauthorized.

McCall's Quality Factor:

This relates to **Integrity**, the system is unable to prevent **unauthorised** access to user accounts. It also aligns with **Testability** as only an external researcher was able to find it, pointing to the poor quality of testing that went into the feature deployment at Meta.

Cause of Defect:

- **Bugs:** Coding error was made while adding the feature to reset passwords, which lacked any brute force protection.
- **Shortcomings of the testing process:** This vulnerability was not found through the security testing process during the development of the feature, a testimony Security testing was flawed.

2.4.5 Good Practices of Change Management for META.

Recognizing that there is no single magic bullet to resolve issues of software quality, certain change management best practices are suggested, based on the discussion of Meta's problems, to prevent Meta from falling into similar problems in the future:

Formal Change Request Process : All changes, big and small should be processed through a formal change request process. This ensures that, before any change is made in the system,

a formal request exists which explains the nature of the change, the reason why the change is required, what the risks of the change are and how the change is to be tested and validated.

Multi-Level Review and Approval : Several people should review and approve any changes prior to applying them. In cases of major changes (e.g., changes to the network configuration or security features) the review and approval should be done by senior engineers and security specialists.

Security Impact Assessment : If there is any modification related to access to the data, authentication or configuration of the system, a formal security impact assessment should be conducted. All potential security risks related to the change should be identified and adequate security controls should be put in place prior to deployment of the change through this assessment.

Mandatory Testing Before Deployment : All changes must be tested in a non-production environment prior to putting it in the live environment. These should be functional tests, security tests and/or performance tests as deemed suitable to the nature of the change being made.

Rollback Planning : The rollout of any change should be accompanied by a well-defined and experimented rollback strategy to be able to undo the change if it creates issues. It's important to test the rollback plan before deployment to ensure it will roll back appropriately.

Change Documentation: Document all changes: what was changed, when the change occurred, who approved the change, how the change was tested, etc. and the outcome of the testing. Having this documentation helps in determining what went wrong after the changed operation is applied.

2.5 SUMMARY OF CHANGE MANAGEMENT ISSUES AND SOLUTIONS

The table below provides a summary of Meta's change management issues and the proposed solutions:

Issue	Year	Change Management Failure	ISO 9126 Factor	McCall's Factor	Proposed Solution
Cambridge Analytica Breach	2018	API change made without proper security review	Functionality Security	Integrity	Mandatory security impact assessment for all API changes
533 Million Data Scraping	2019	Feature deployed without testing for automated abuse	Functionality Security	Integrity, Reliability	Security testing to include automated abuse scenarios
Global Platform Outage	2021	Configuration change applied without testing or rollback plan	Reliability Recoverability, Maintainability	Reliability, Maintainability	Multi-level approval and mandatory rollback planning for all configuration changes
Account Takeover Vulnerability	2024	Password reset feature deployed without brute-force protection testing	Functionality Security	Integrity, Testability	Mandatory security testing for all authentication features before deployment

2.6 THE MODEL DEVELOPED FOR CHANGE MANAGEMENT FOR META :

Having analyzed the problems of Meta, it is highly recommended to adopt a structured and formal change management model. This model should contain the following main phases for changes in Facebook platform:

Stage 1- Change Identification :

Any change to an application (e.g. feature, bug fix, security patch or configuration change). A full description of the change is put forward in a formal change request.

Stage 2 - Change Assessment :

Defines how much the change might affect the system. This involves risk assessment, a security impact assessment and an estimate of the resources and time the change will require.

Stage 3 - Change Approval :

The change request is approved by the team leads, security and more senior engineers. It will only change if it conforms to all the needed standards and if all the identified risks have been resolved.

Stage 4- Change Implementation:

Change is put in place in a controlled development environment. Changes in coding should be in accordance with coding standards and coding documentation guidelines.

Stage 5-Change Testing :

You will comprehensively test the change in a non-production environment. Take into account functional, security, performance and regression testing, where applicable.

Stage 6 -Change Deployment :

A controlled and monitored deployment of the tested and approved change to the live production system. Rollback plans should be available and tested prior to deployment.

Stage 7-Change Review :

Once the change is deployed, it is monitored for a specified period of time to ensure that the change is functioning properly. Any issues that are discovered at this time are given a priority and the Rollback Plan is initiated, if required.

If applied correctly to the various changes to Meta's Facebook platform over the last couple of years, the risk of such software quality failures encountered by Meta would be much lower.

3.VERSION MANAGEMENT

3.1 VERSION MANAGEMENT INTRODUCTION

Version Management is one of the key features of Software Configuration Management (SCM), which enables the control, tracking and management of changes to software systems over time. Software evolves, in improvements introduced, defects fixed, security issues addressed and performance optimized. All of these changes result in another version of the software which needs to be documented and managed.

In big tech companies such as Meta, thousands of code changes are made every day by different development teams. It's difficult to determine the source of issues, keep software stable and current and to coordinate development in the absence of a good version management system. Hence, version management is a crucial factor to assure software quality, reliability and security.

3.2 VERSION MANAGEMENT'S OBJECTIVE

Listed below are version management's primary objectives:

- To keep a complete history of the software changes.
- To allow previous versions of the software to be saved for future reference.
- In order to facilitate interaction between several developers.
- To minimize mistakes and issues during a development.
- To provide rollback choices to a situation of a failure.
- To enhance software reliability, maintainability and quality.
- To only deploy Approved and Tested Versions.
- Version management plays a crucial role in software quality assurance by ensuring that various software versions are readily available for use.
- Version management is essential for Software Quality Assurance (SQA) as it makes sure that any modifications that are made to the software are well controlled and documented.

It will be able to accrue the following:

- Complete change history present.
- More convenient to catch and repair computer program faults.
- Tie-in to restoring to earlier stable versions as need be.
- Better Development Team Facilities.
- Greater control of software releases/updates.
- Uncovering unauthorized or unauthorized changes.
- Improved auditability and documentation.

3.4 META'S SOFTWARE QUALITY ISSUES EMERGING DUE TO POOR VERSION MANAGEMENT

3.4.1 ISSUE 1: CAMBRIDGE ANALYTICA DATA BREACH (2018)

Version Failure:

Meta attempts to manage and oversee API version and permissions alterations. With the lack of API update management, third party apps have had access to user information more than they had permission.

Impact:

- Unauthorized access to user information.
- No suggestions
- Security and privacy issues.
- Increased damage to the reputation of others.
- Penalties and regulators.

Solution :

- Tightly managed API version control.
- Make API change approval processes more official.
- Prior to each release: security checks.

3.4.2. ISSUE 2: Incident No. 34219 User Data Scraping Incident.

Version Management big fails include:

There was a significant increase in the Contact Importer feature which had not been sufficiently tested, validated, or version tracked.

Impact:

- Information of customers is accessible.
- Increased privacy and security threats.
- Issues of compliance and regulatory fines.
- Loss of user trust

Solution:

- Conduct pre-deployment testing as required.
- Maintain a detailed version of records.
- Put in place controlled release management procedures on procedure.
- Testing for security of all software versions.

3.4.3 ISSUE 3: GLOBAL PLATFORM OUTAGE (2021)

Version management failure: Widespread service outage due to incorrect version of the network configuration being set-up in the production environment.

Impact:

- 6 hours with the service of the issue.
- Facebook, Instagram and WhatsApp services are disrupted.
- Large monetary and operating losses.
- Reduced customer confidence.
-

Solution:

- Set up frequent use of configuration version control systems.
- Use automated deployment verification .
- Implement rollback plans to release when they fail
- Approval is key prior to production deployments.
- In 2024, it was released that the system has a password resetting vulnerability.

3.4.4 ISSUE 4 FAILURE OF VERSION MANAGEMENT

The reset password feature was exposed if one was not tested and validated for security.

Impact:

The following risks of an unauthorized access to your account are greater:

- Potential exploitation potential attackers.
- Platform vulnerabilities.

Solution proposed:

- Thoroughly test for security before releasing.
- Establish procedures of version verification.
- Use strategies to control and easy deployment.
- Not all features are available due to security restrictions.

3.5 META'S BEST PRACTICES FOR VERSION MANAGEMENT

To minimize the amount of risk faced during operation and to enhance software quality, the following version management practices should be adopted by Meta:

- Version Numbering Standard
- Structured Version Numbering System should be released each release of the version. example:
    - Version 1.0: Initial Release
    - Bug
    - Major Feature Release – version 2.0 has been released.
    - Security Update, Version 2.1

- Areas of responsibility for the Centralized Repository.
- All the source code and configuration files should be placed in a centralized location, for consistency and ease of access.

Recommended tools:

- Git-version control system for tracking code changes locally.
- GitHub-Cloud platform for hosting Git repositories and collaboration.
- GitLab-DevOps platform for Git repository management and CI/CD automation.

Concept of Branch Management Strategy

Various types of branches should be kept for their various activities of development:

- A Main Branch
- Branch Development
- Feature Branch
- Branch in Testing
- Branch Hotfix

This method helps with parallel development and decreases conflicts.

Tracking and Documentation Change

The detailed records of every software version should be provided, e.g.

- Change description
- Information of developer
- Date of modification
- Approval status
- Test results

This safeties and secures the traceability and accountability as correct documentation.
All versions should be Code Review prior to deployment.

- Security Review
- Review Quality Assurance
- Management's approval

This can guarantee that the versions that are released are the ones that have been validated.
Capability Rollback: If a newly deployed version causes failures or security issues, organizations should be able to immediately restore previous stable versions.

3.6 META RECOMMENDED VERSION MANAGEMENT MODEL

Here is the recommended procedure to be followed when using version management:

Stage 1: Creation Version
Developers are able to build new functionality, improvements and fixes.
Stage 2: Registration Version
The modifications get registered and stored to the version repository.
Stage 3: Review Code
Senior Developers conduct a functionality check, as well as a standards and code quality check.
Stage 4 : The version goes through:

- Functional Testing
- Testing security
- Performance Testing
- Regression Testing

The final stage is when the published version is approved.
QA teams and management approve release, after reviewing the results of tests conducted.
Stage 5: Release Version
The approved version is used to build up the production environment.
Stage 6: Monitoring versions
The system is evaluated both in terms of its performance and users feedback on a regular basis after its deployment.

Stage 7: Rollback version

If critical problems have been found, Rollback Version–stage 8 rolls back to the last stable one.1984 Not meeting their needs Management failure Language/situational mismatch New Partnership Compact, with alternate language plans, seeks to resolve issues.

Cambridge Analytica Breach 2018 Poor API version security API approval, security review. Data Scraping Incident 2019 improper testing and tracking Security, Reliability Version Testing Mandatory.

Rollback and configuration control. Wrong configuration reliability deployment 2021 global platform outage.

Password Reset Vulnerability 2024 improper Security Validation Security Security testing before release.

3.7 CONCLUSION

Having version management is crucial for large-scale organizations like Meta to ensure the integrity, security and stability of their software solutions. and with the aid of effective version control, teams can keep a record of software modifications, control launches, boost teamwork and swiftly recover from failures. Poor version management procedures can end up as operational risks, service disruptions and security issues and that has shown by Meta.introducing a Git-based version management system, standardized process strict test practices approval procedures and rollback capabilities, Meta can significantly increase software quality assurance, improve security and reduce the chance of recurrent events.

4. SYSTEM BUILDING BY PROVIDING THE INTERFACE(S) OF THE SYSTEM BUILD

4.1 OVERVIEW OF THE SYSTEM

The proposed system is a Facebook Clone Web Application that provides the users with the core social networking functionalities like registration of accounts, secure login, post sharing, friends management, messaging, notifications and profile management. The goal of the system is to simulate a familiar social media environment with a strong focus on software quality standards.

The system interfaces were designed in Figma before implementation in order to visualise the structure of the system, validate the user requirements and improve communication among the members of the development team. This early prototyping approach helped to discover usability issues at an early stage and largely reduced the re-design effort during the development.

Figma Prototype Link:

https://www.figma.com/design/6Ml2UgiSTdDEt3Sjcc3pXL/Software_Quality_Group_project?node-id=20-4&t=PvpTJJUhgsq7jsRh-1

The design process focused on key software quality attributes such as usability, reliability, integrity, efficiency, maintainability and portability. This was to make sure that the system is secure, responsive and user friendly.

4.2 SYSTEM ARCHITECTURE OVERVIEW

The Facebook Clone is built based on a modular client-server architecture, where system features are divided into separate modules. This is suitable for scalability, maintainability and future flexibility.

SYSTEM ARCHITECTURE DIAGRAM

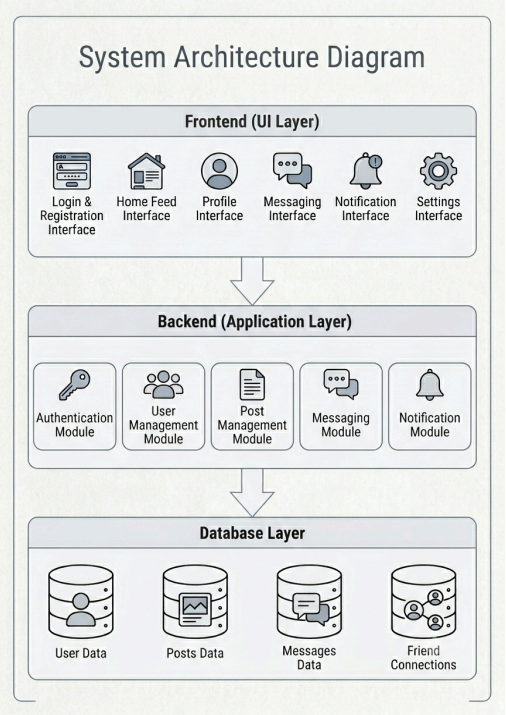


FIGURE-1: SYSTEM ARCHITECTURE OF FACEBOOK CLONE

SYSTEM MODULES

- User Authentication Module
- User Registration Module
- Home Feed Module
- Profile Management Module
- Friend Request Module
- Messaging Module
- Notification Module
- Settings Module

USER FLOW

The system begins by completing Registration/Login and then proceeds to the Home Feed where users can interact by means of posts, comments and friend requests. They can then go to profile management, messaging, notifications and settings and sign out securely.

4.3 INTERFACE DESIGN

4.3.1 LOGIN PAGE

Interface Description

Login Page: Secure login access for existing users using email/phone and password authentication.

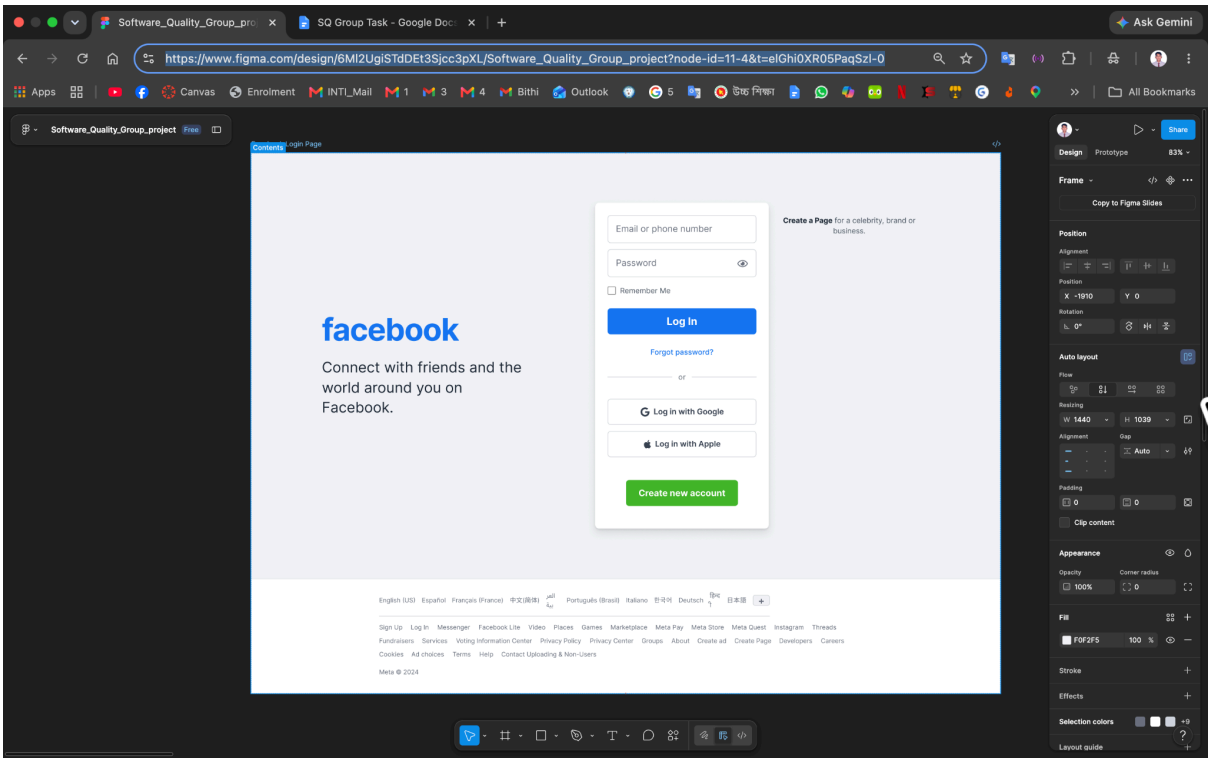


FIGURE-2: LOGIN PAGE UI

Software Quality Justification

The login interface has a clean and minimal layout to improve usability and reduce cognitive load. Password masking is a security and integrity feature that prevents unauthorised viewing. Additionally, efficient form verification simplifies the authentication process, reducing the login time and system overhead.

4.3.2 REGISTRATION PAGE

Interface Description

In the Registration page a new user creates an account and provides the credentials and personal information.

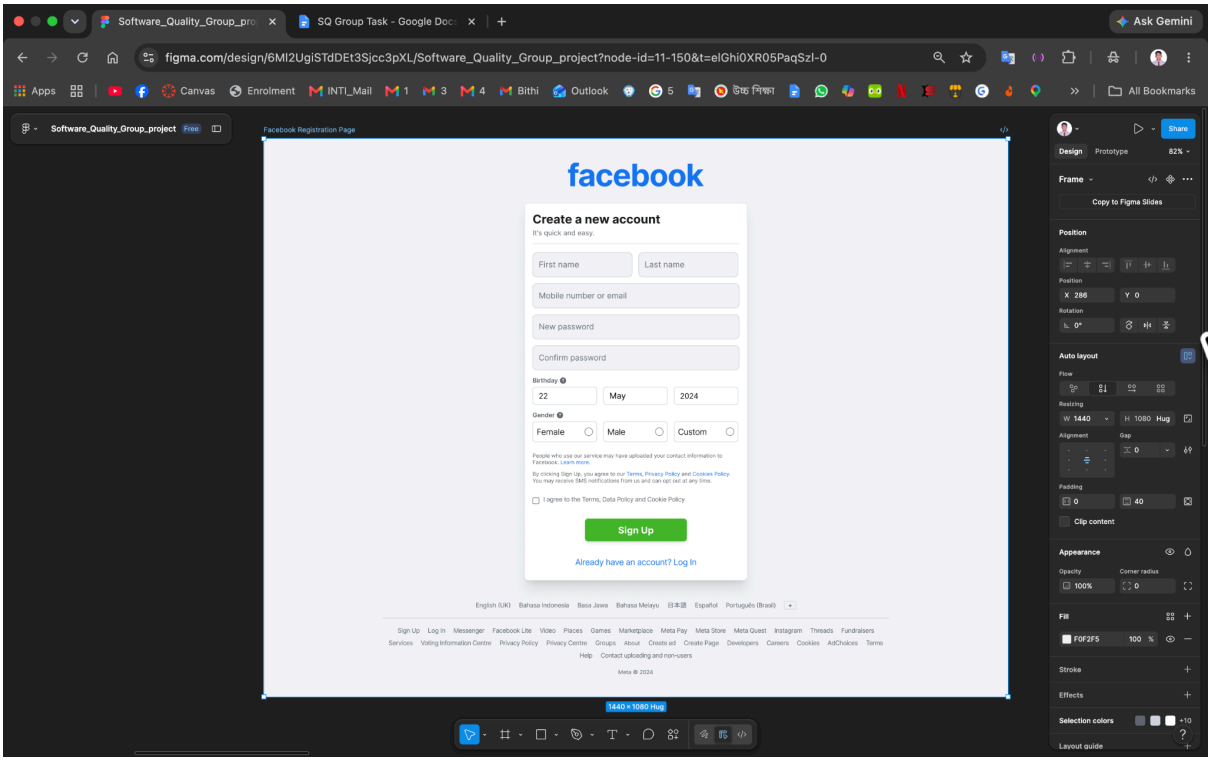


FIGURE-3: REGISTRATION PAGE UI

Software Quality Justification

Input validation ensures the correctness of user data therefore resulting in less invalid or incomplete submissions. The structured form layout is a good usability standard to lead users step by step. Increased password strength requirements improve system integrity and security against weak credentials.

4.3.3 HOME FEED PAGE

Interface Description

The Homepage Feed is the main interaction area where users may view, create and interact with posts.

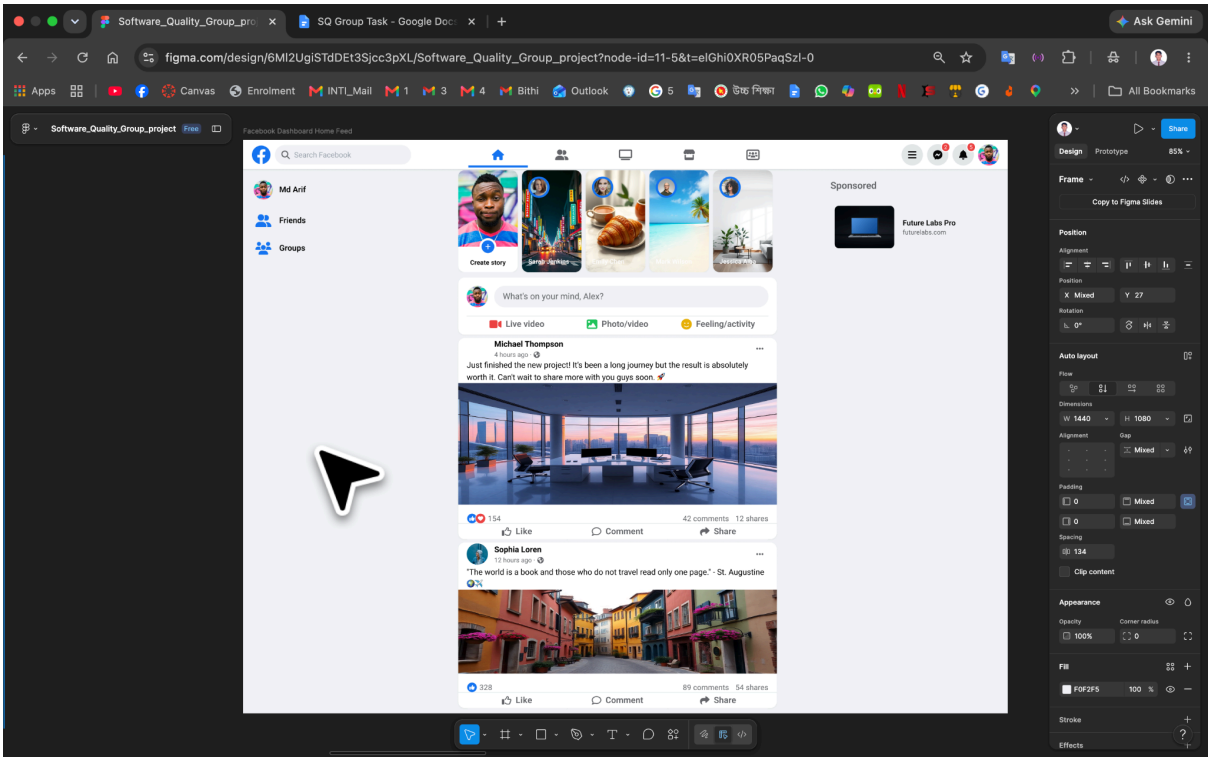


FIGURE-4: HOME FEED UI

Software Quality Justification

The interface is built for fast rendering of content to improve system performance. The centralised feed structure provides a uniform way to display data that enhances reliability. The known social media interface improves usability and decreases the time it takes to learn the system.

4.3.4 USER PROFILE PAGE

Interface Description

User Information, Posts and Social Connections displayed on the Profile Page.

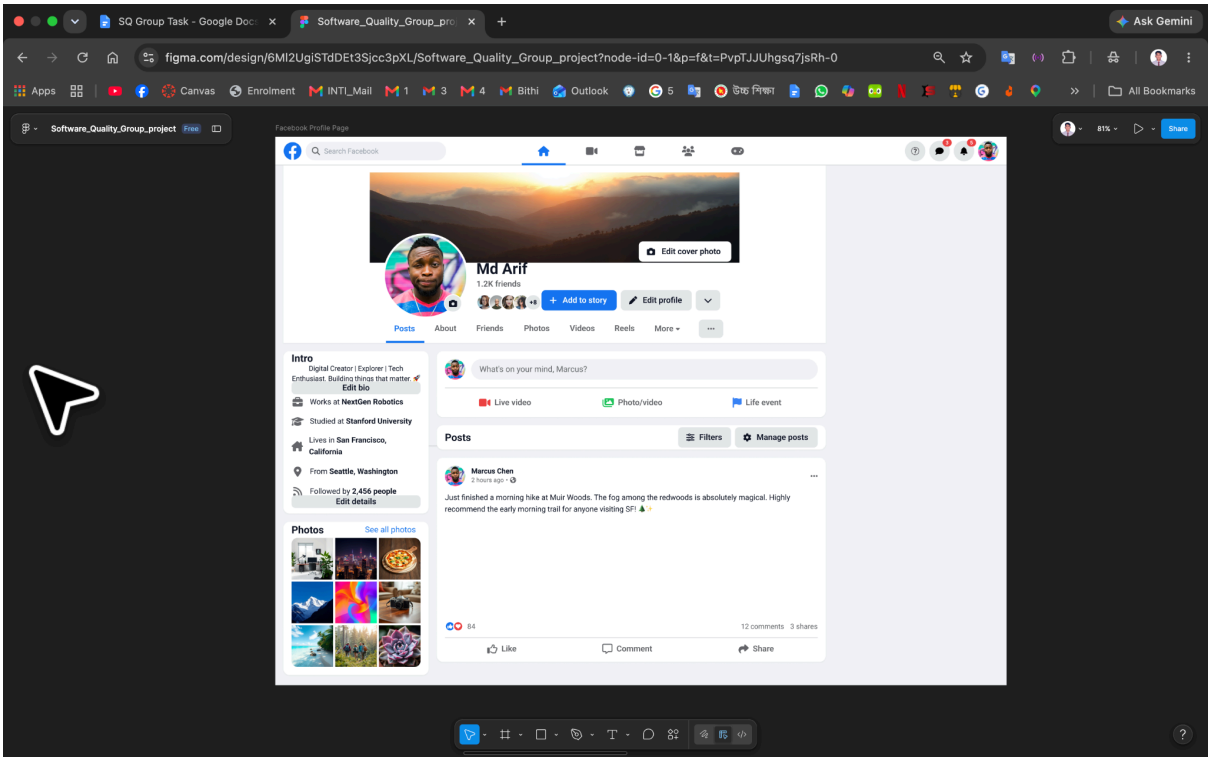


FIGURE-5: USER PROFILE UI

Software Quality Justification

Accurate retrieval of user data is important for the correctness of displayed information. Modular profile components make updates easy and improve maintainability. Well-defined layout structure can enhance the usability and navigation efficiency.

4.3.5 FRIEND REQUEST PAGE

Interface Description

Users can send and receive friend requests and suggestions through this interface.

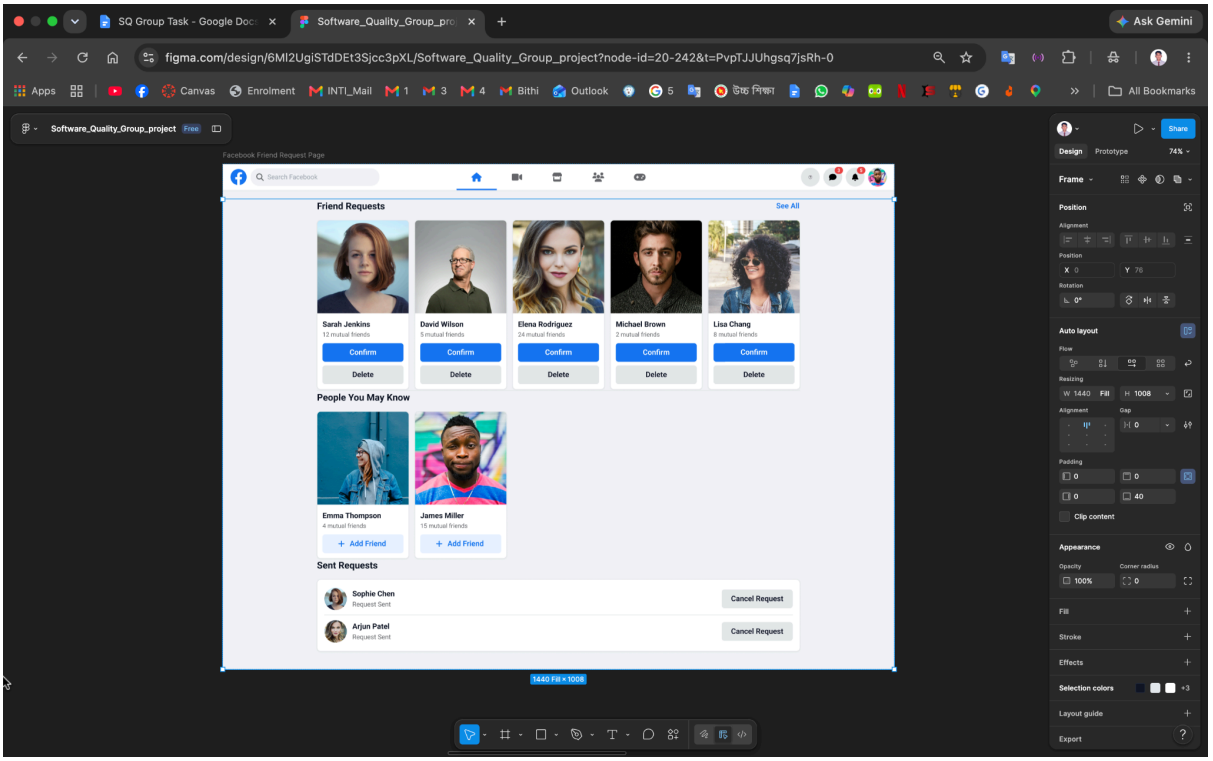


FIGURE-6: FRIEND REQUEST UI)

Software Quality Justification

Well positioned action buttons improve usability and speed up user decision making. Processing requests efficiently keeps the system responsive and real-time updates ensure data accuracy and consistency.

4.3.6 MESSAGING PAGE

Interface Description

The Messaging Page allows users to communicate in real time.

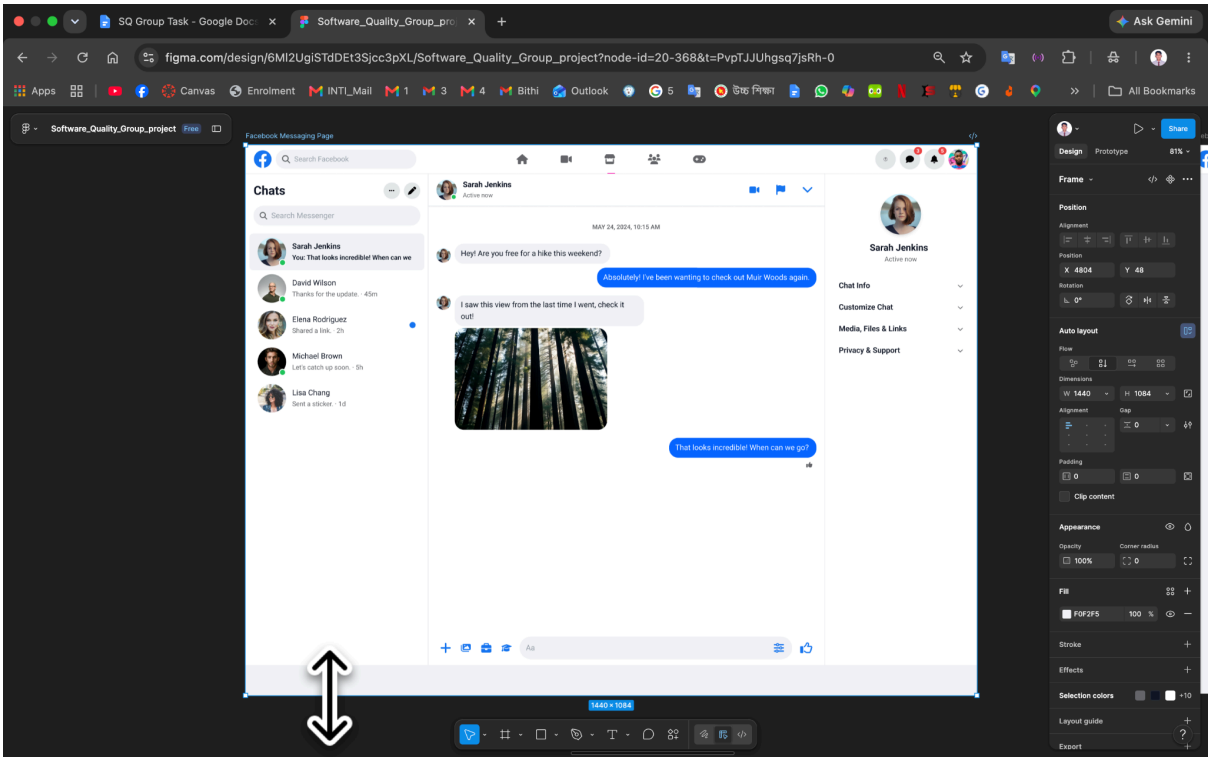


FIGURE-7: MESSAGING UI

Software Quality Justification

Real-time message synchronisation enhances the reliability of the system. Fast message transmission enhances efficiency. Secure data handling ensures the integrity of communication.

4.3.7 NOTIFICATION PAGE

Interface Description

Notification Page: The page that shows system alerts and updates on user activity.

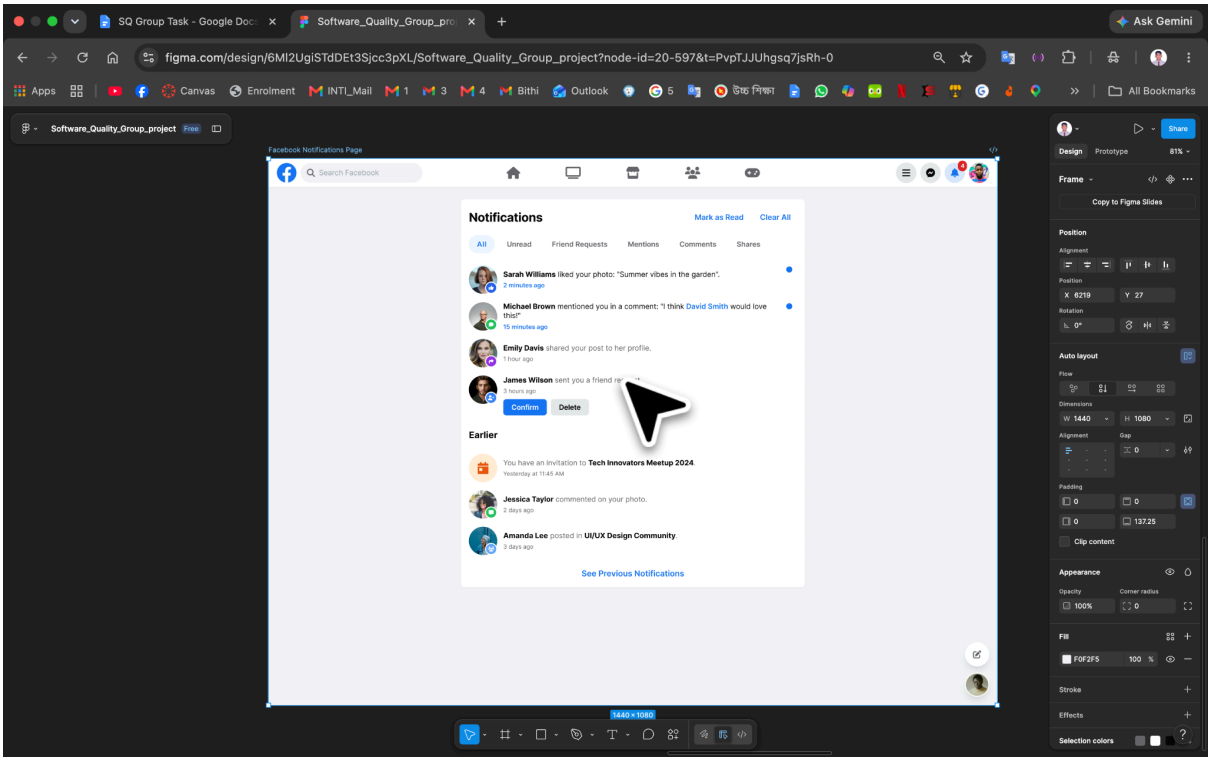


FIGURE-8: NOTIFICATION UI

Software Quality Justification

The real time notification updates ensure information is accurate and up to date. Organization is structured for usability and rendering is optimised for system efficiency.

4.3.8 SETTINGS PAGE

Interface Description

In Settings Page Users can manage account, privacy and security preferences.

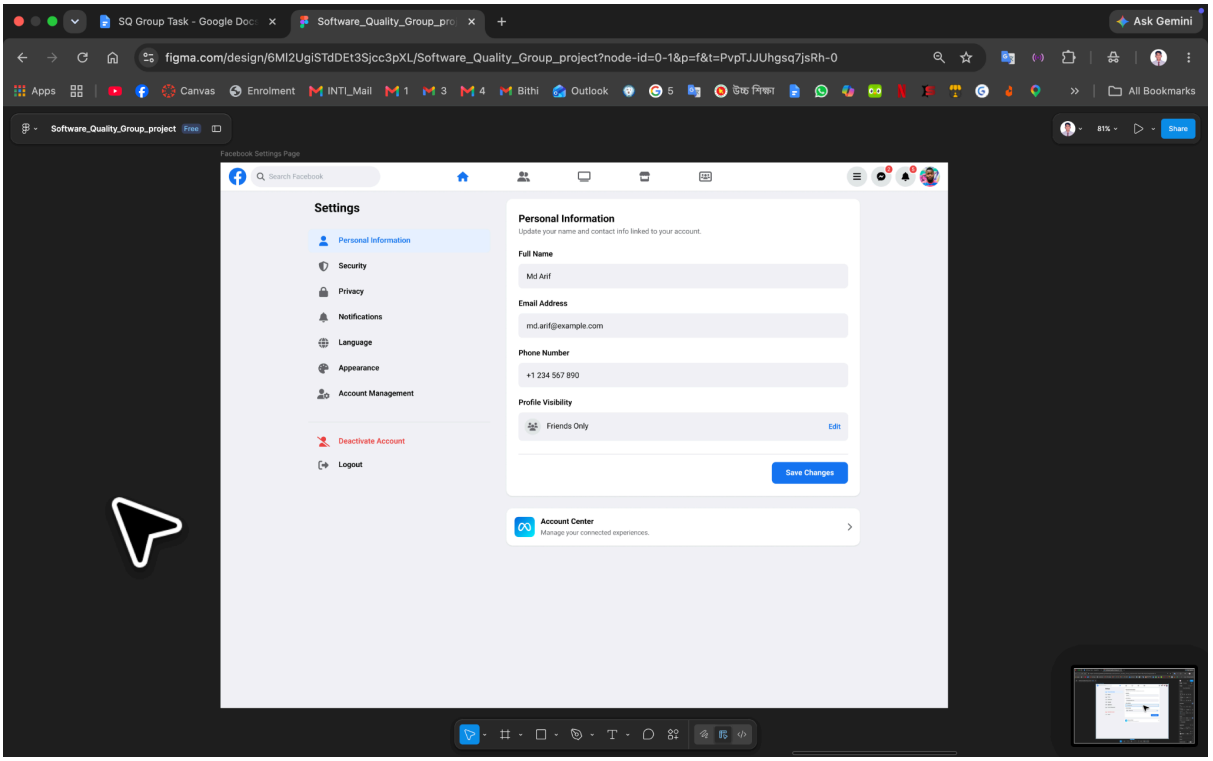


FIGURE-9: SETTINGS UI

Software Quality Justification

Centralised control makes it easier to use and maintain. The privacy and security settings make sure that the system is working properly . The organised options help to reduce the confusion of the users and make it more accessible .

4.4 SOFTWARE QUALITY IMPROVEMENTS ACHIEVED THROUGH THE DESIGN

QUALITY FACTOR	IMPLEMENTATION IN FACEBOOK CLONE
CORRECTNESS	Input validation and accurate data rendering
RELIABILITY	Stable authentication and messaging system
EFFICIENCY	Optimized navigation and fast page loading
INTEGRITY	Secure login, password protection, privacy controls
USABILITY	Familiar Facebook-like interface design
MAINTAINABILITY	Modular architecture with independent components
PORTABILITY	Responsive design for desktop and mobile devices

4.5 SYSTEM WORKFLOW SUMMARY

In order to clearly depict system interfaces and guarantee consistency with user needs prior to deployment, the Facebook Clone prototype was created using Figma. In order to ensure a stable and user-friendly system, the design places a strong emphasis on important software quality attributes including usability, dependability, integrity, efficiency, maintainability, correctness and portability.

The system provides a consistent and user-friendly social networking experience while supporting simple future development and maintenance thanks to its modular design and well-organised interfaces. The prototype provides a solid basis for system development by successfully bridging the gap between design and implementation.

5. RELEASE MANAGEMENT

Change the software as a controlled process of organizing, testing, approving and accomplishing the software changes is known as release management. It helps make sure that tested and well founded software is provided to the production environment.

Digital Connect Solutions search was undertaken due to customer service platform's seeming issues with payment failures, system crashes, poor performance, security faults and integration challenges. Because of these problems it is certain that the firm did not have an appropriate release policy for software confirmation and deployment.

Failure to give the services requested, errors in payments, manual Get inside Recovery and make the system less reliable, decrease the consumer satisfaction degree, boost the support charge and result in reputational damage, are traits of on poverty stricken service release management. Safe, reliable and traceable releases which satisfy the business need and reduce deployment risks, improve the performance of the systems, support business continuity, improve efficiency and do not harm management processes.

5.1 PROBLEMS IDENTIFIED

Raises a number of case release management issues.

- Deployment process (inconsistent understanding of releases). No uniform procedure was followed in evolution and delivering release.
- Software components were Mistakenly tracked and documented, which resulted in the wrong versions being released.
- To production without ensuring carry through threatening changes unauthorized systems were provided to production.
- Inadequate testing occurs preceding releases, especially in performance, security and reproduction production workload conditions.
- Incapable rollback planning may have hampered the organization's potential to quickly recover from futile releases.

These defects demonstrate that the company wasted money to oversee a “release”, as a one-sided operation – editing, validation, approval, release and monitoring – and as only the last phase of development.

5.2 COMPANY RELEASE AND THE IMPORTANCE OF RELEASE

MANAGEMENT IN THIS CASE

The term release" is a key component to this instance of a company.

It's release management that's required for mitigating the waterfall crash of impoverished software. If the organization had brought several of the similar release administration concepts through, many of the same system problems could have been prevented prior to the platform becoming offered to customers.

The principle Release management is applicable in these situations due to the following reasons:

- To prevent production problems, ensure that every change is authenticated and tested previously being executed.
- To promote trust, staying away from being dishonest releases contract services or personal information.
- Enhance the flow of information among coders, testers, ops personnel, security groups and business users.
- To guarantee company continuity, deployment plans do not upset censorious services such as payments and client hold-up.
- Maintaining traceability documentation so that the business knows what was changed, by whom and when it was made available for use.
- to allow fast recovery time if deployment is associated with problems.

The release management is a technical authority and also a commercial protection compute for a cloud based platform, like DigitalConnect Solutions.

5.3 PROJECT HAIR COLOURATION RECOVERY PLAN

DigitalConnect Solutions should put in place an organized, release management approach in order to enhance the platform. Phases should involve different aspects, from the planning stage to the stage of post-release evaluation.

5.3.1 PLANNING FOR RELEASE

The company should create a release plan prior to releasing any releases. The plan should define the scope, the modules affected, risks, timeline, the team members responsible, testing requirements and deployment processes.

The release plan should have the following:

- The release intends to comprise an outline of the features and fixes added to the system, such as security holes, system performance and payment issues.
- Interdependencies between system modules and other corporate systems.
- Blocked summer policy not to be deployed.

- Security evaluation of critical modules such as payment, login and some of the client databases.
- Rollback procedure in such case the deployment fails.

5.3.2 BUILD AND PRE-RELEASE VALIDATION

This prepares each build of software in a sandbox-type environment. Build must pass validation stages before moving on to a release to ensure that it is production ready.

The following should be checked in the Build:

- Testing is conducted to ensure that all communication technologies, registration of accounts, payment systems and requests for services will work.
- To guarantee that the new system with databases and corporate systems which are already in place, integration testing is important.
- Make sure the platform can handle high user traffic without evaluating its performance.
- Security testing to identify problems to data revelation, issues regarding access control and feebleness in authentication.
- User acceptability testing (UAT) is designed to determine whether the release meets with users' and business' expectations.
- Regression testing: testing that is designed to make sure that the new adaptations don't impact an already functional one.

5.3.3 RELEASE APPROVAL PROCESS

There ought to be a man approval process to deployment. Unnecessary risks were incurred due to unauthorized changes to the case study. A remedy to this is necessary, which can be done by a review by responsible stakeholders on every release.

The approval group could consist of:

Role	Accountability
Project Manager	Evaluates the Timetable, business effect and release readiness.
SQA Team	Verifies that testing is finished and that quality criteria are fulfilled.
Security Team	Confirms vulnerability patches and security tests.
Configuration Manager	Verifies accurate documentation and versions
Operations Team	Evaluates environment stability and deployment readiness.

Through this release process, no release without technical and business verification is put into production.

5.3.4 CONTROLLED DEPLOYMENT

Before the actual deployment, a standard process should be adhered to. Instead of bettering the live system without control, the organization should choose a phrased rollout technique.

Examples of controlled deployment include:

- Pilot release: initial release for a limited number of individuals.
- Phased rollout is a strategy that involves a gradual Introduction of the product over a period of time.
- Blue-green deployment: Keep two production environments, then only route traffic to the new environment as soon as it's attested to be stable.
- Canary release: gradual deployment to a small sample of real users to see system behaviour.
- Lastly, a canary release / staggered deployment would be fine for given that
- Digital Connect Solutions handles substantial client transactions and payments. This minimises danger and the team looks for bugs before they can impact all users.

5.4 PROPOSED RELEASE WORKFLOW

The Diagram below shows the company's release workflow.

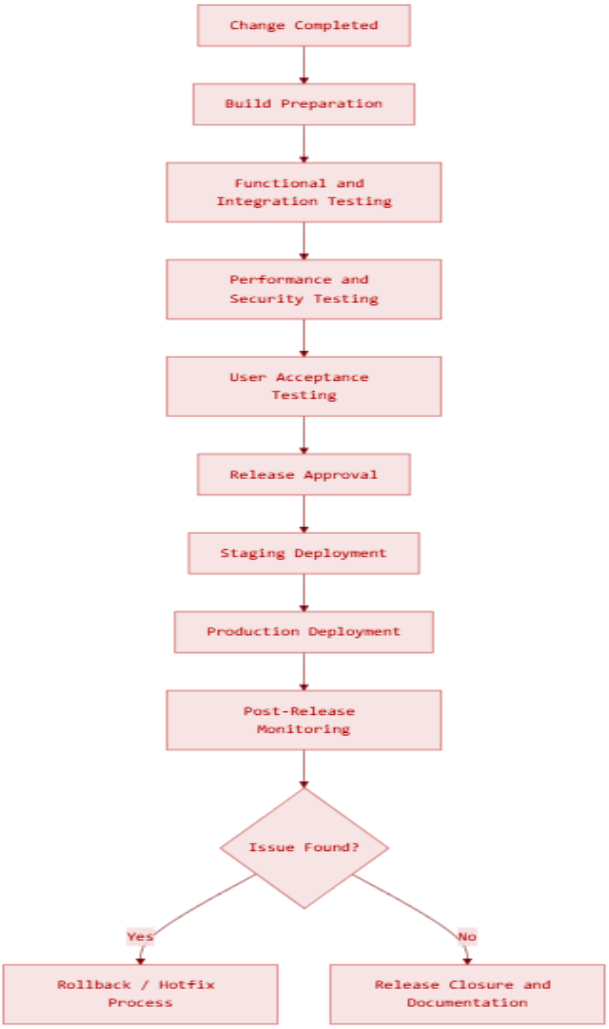


FIGURE-10: COMPANY'S RELEASE WORKFLOW

Before releasing ensure that it works as per that workflow, so then take out the chance that it may affect customers.

5.5 ROLLBACK AND RECOVERY PLAN:

Below are some of the things that can be done after it has been released:

- Tracking how well the system is working by looking at how busy the servers are, how fast it responds and how quickly webpages load.
- Monitoring the number of requests that are not successful monitoring how many mistakes are made with payments. How many problems there are for people to log in.
- Checking alerts to determine if a system has any vulnerabilities or if someone is attempting to access the system in an unauthorized manner.

- Analysing support questions and customer complaints to identify any issues that are impacting customers.
- Monitoring integration logs to ensure that information is integrated properly between systems.

This helps the company to react faster before a small issue becomes a problem.

5.6 RECOMMENDATION FOR THE IMPROVEMENT OF RELEASE MANAGEMENT FOR THE IDENTIFIED ISSUES:

The case has issues with the quality of the software and each one can be helped by better release management.

- Issues with software quality and how to improve release management.
- Until all of the required testing is complete, the system shouldn't be made available to people requesting and paying for it, because according to our tests, it crashes.
- When the system is being used a lot it is slow so load and performance tests should be required before it is approved.
- For mistakes with payment transactions a plan should be made to reverse the changes. The release should be done in stages.
- In case of such deficiencies in security it is strongly recommended to conduct a security review and approval prior to the system release.
- The integration testing should be performed prior to release when the problems with database synchronization occur. The data should be checked to make sure it is consistent.
- If the UX is not good, UAT needs to be performed prior to modifying the user interface.

This proves helpful for the quality and stability of the business because of release management.

5.7 SAMPLE FOR A DIGITAL CONNECT SOLUTIONS RELEASE PLAN:

The following could be a plan for releasing a significant update:

Description of the Item:

Known Issues in this Release:ávalagsmart.com/wiki (client)

Goals: Payment issues, make them faster, authentication issues and pushing things to be synced.

Modules: Database integration services, payment, login and customer request.

All the necessary tests, including functional, regression, integration, performance, security and user acceptance testing should be done.

Deployment Strategy: The release should be done in phases starting with a test group.

Rollback Trigger: When there is a serious crash in the system or a security alert or a payment failure is beyond the allowed amount.

The approval of the Project Manager, SQA Team, Security Team, Configuration Manager and DevOps Team are needed.

Such a release plan renders the deployment process more organized and professional.

5.8 THE BENEFITS OF A GOOD RELEASE MANAGEMENT PROCESS:

Digital Connect Solutions will receive benefits if it releases management rights.

The system is more stable as only properly tested releases are sent to production.

There are nice such things as huge failures which contribute to the consumer experience.

5.9 RELEASE MANAGEMENT CONCLUSION:

To sum up release management is an important part of software quality management, for Digital Connect Solutions. But the company was plagued by poorly executed tests, substandard approval protocols and substandard release control. Therefore, customers were not happy: there was no software stability and it was not secure software that was being released into the environment and thus operational inefficiency and loss of finances resulted from this.

The company should have release management that involves the capability to roll back, controlled deployment, extensive pre-release testing, formal approval process and ongoing monitoring and management of the post-release environment. In doing this Digital Connect Solutions can minimize the dangers associated with releases, enhance the reliability of its services and avert software problems in the future. A solid release management plan also helps to increase the delivery of software updates safely, strengthen customer trust and help in long-term growth strength of a business.

6. CONCLUSION

The present report discussed the software quality problems that Meta Platforms, Inc. experienced in the Facebook platform, which runs on the cloud, after 2018. 4 real life and publicly documented software quality failures were selected, analysed and put in aspects of the software quality frameworks. Based on the proper software quality assurance practices, improvement strategies were proposed.

6.1 THIS REPORT DEALS WITH FOUR ISSUES:

The Cambridge Analytica Data Privacy and Security Breach (2018) is a leading issue.A leading issue is Cambridge Analytica Data Privacy and Security Breach (2018).533 Million User Data Scraping Vulnerability (2019-2021) is the issue.There were no blackouts in other regions.Facebook Account Takeover Vulnerability in Password Reset System (2024) issue No.4 has been released.

6.2 A SUMMARY OF THE KEY FINDINGS:

Software failure seriously impacts the business operations. Each of the problems noted in this report brought about repercussions that went well beyond the loss of an apparatus. The FTC levied a fine of $5 billion as a punitive measure after the Cambridge Analytica incident. The outage in 2021 took a cost of $60 million in lost revenue. They became the target of a 533 million data scraping scandal with a subsequent GDPR fine of €265 million. These facts and figures indicate that poor software quality is not a purely technical issue – it is a business issue. This is a direct case in point for the domino effect – when one software quality failure sends repercussions across the entire organisation.

The majority of failures could have been avoided. In every case, the reason for the problem was the lack of adequate testing of change prior to deployment, a failure to consider security implications or unsuccessful rollbacks that lacked recovery options. Proper SQA practices, change management and version management are all directly addressing all of these.

Each of the nine possible causes for software defects was clearly evident. The problems that were common across all four were some deficiency in the testing process, especially in the lack of proper security testing prep before launching new features and changes.

Several ISO 9126 quality attributes were not adhered to. A Security failure was found in 3 out of 4 issues related to Functionality. It was clear there were reliability failures during the outage in 2021. Recovery was delayed unduly due to a failure in maintainability. This indicates that all the six software quality factors need to be considered simultaneously.

Meta has developed over the years. Meta had more and more controls, tests and processes after every major failure. This reflects a willingness to continuous improvement – the goal of Software Quality Assurance.

6.3 KEY RECOMMENDATIONS

- Make security testing mandatory for every new feature and change, not just at the end implement this change to enhance security testing.
- Adopting proper change management : All changes should be subject to a change identification, evaluation, approval, test, deployment and review process before being deployed to the production system.
- Version-control everything : Network configurations and infrastructure settings should be version-controlled just like software code and should have built-in abilities for automated rollback.
- Ensure that independent recovery systems are in place: Recovery and monitoring tools are installed on a separate network - and they are entirely separate from the production system - so that engineers can always be able to use them, even when the primary system fails.
- Regular security audits : Completing comprehensive security audits on a regular basis, conducted by in-house personnel and outsiders.
- Improve requirements definition : Document/define all security, performance and privacy aspects required prior to development to minimize defects up front.
- Implement Agile development officially : Agile is the right model of development for Meta as it enables continuous development, inbuilt testing and quick fixes for security and quality issues.

6.4 CLOSING STATEMENT

Meta's Facebook platform has almost 3 billion people using it each day. With such a critical amount of software to manage, it is a massive task! The issues that plagued the company from 2018 to 2024 harmed users, the company and the brand's reputation. But these failures also had some important lessons.

Quality is conformance to requirements — any non-conformance is a defect. It also involves fitness for use, that is, it should satisfy the needs of the users and make them feel better. Meta's users demand that everything is operational, secure and accessible at all times. That means that SWQA, change management, version management and release management must be embraced 100% throughout the software development process.

By implementing effective strategies and protocols, Meta can minimize downtime, safeguard user information and ensure a seamless and safe experience for their users globally.

7. REFERENCES

1. Angwin, J., Larson, J., Mattu, S., & Kirchner, L. (2016, March 1). *Machine bias*. ProPublica. https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing

2. Cadwalladr, C., & Graham-Harrison, E. (2018, March 17). *Revealed: 50 million Facebook profiles harvested for Cambridge Analytica in major data breach*. The Guardian. https://www.theguardian.com/news/2018/mar/17/cambridge-analytica-facebook-influence-us-election

3. Confessore, N. (2018, April 4). *Cambridge Analytica and Facebook: The scandal and the fallout so far*. The New York Times. https://www.nytimes.com/2018/04/04/us/politics/cambridge-analytica-scandal-fallout.html

4. Datadog. (2021, October 5). *Facebook outage post-mortem*. Datadog Blog. https://www.datadoghq.com/blog/facebook-outage-oct-2021/

5.Davies, R. (2022, November 28). *Meta fined €265m over Facebook data breach affecting 533 million users*. The Guardian. https://www.theguardian.com/technology/2022/nov/28/meta-fined-265m-over-facebook-data-breach-affecting-533-million-users

6. Federal Trade Commission. (2019, July 24). *FTC imposes $5 billion penalty and sweeping new privacy restrictions on Facebook*. Federal Trade Commission.

https://www.ftc.gov/news-events/news/press-releases/2019/07/ftc-imposes-5-billion-penalty-sweeping-new-privacy-restrictions-facebook

7. Federal Trade Commission. (2023, May 3). *FTC proposes blanket prohibition preventing Meta from monetizing youth data*. Federal Trade Commission. https://www.ftc.gov/news-events/news/press-releases/2023/05/ftc-proposes-blanket-prohibition-preventing-meta-monetizing-youth-data

8. Feitelson, D. G., Frachtenberg, E., & Beck, K. L. (2013). *Development and deployment at Facebook*. IEEE Internet Computing, 17(4), 8–17. https://people.engr.tamu.edu/pcr/courses/csce431/fall19/reading/FacebookDevelopment.pdf

9. Franceschi-Bicchierai, L. (2021, April 3). *Someone had already leaked the data of 533 million Facebook users for free online*. Vice. https://www.vice.com/en/article/someone-had-already-leaked-the-data-of-533-million-facebook-users-for-free-online

10. Greenberg, A. (2021, October 4). *Facebook's outage reveals the internet's dangerous power concentration*. Wired. https://www.wired.com/story/facebook-outage-internet-power-concentration/

11. Hautala, L. (2022, November 28). *Meta fined $275 million for data scraping that exposed 533 million Facebook users*. CNET. https://www.cnet.com/tech/services-and-software/meta-fined-275-million-for-data-scraping-that-exposed-533-million-facebook-users/

12. Hunt, T. (2021, April 4). *Have I been pwned and the Facebook data*. Troy Hunt Blog. https://www.troyhunt.com/have-i-been-pwned-and-the-facebook-data/

13. Ireland Data Protection Commission. (2022, November 28). *Data Protection Commission announces conclusion of inquiry into Meta Platforms Ireland Limited*. Data Protection Commission. https://www.dataprotection.ie/en/news-media/press-releases/data-protection-commission-announces-conclusion-of-inquiry-into-meta-platforms-ireland-limited-2022

14. Jankowicz, M. (2021, October 5). *Facebook says its 6-hour outage was caused by a faulty configuration change on its routers*. Business Insider. https://www.businessinsider.com/facebook-says-outage-caused-by-faulty-configuration-change-on-routers-2021-10

15. Kastrenakes, J. (2021, October 4). *Facebook, Instagram and WhatsApp are all down*. The Verge. https://www.theverge.com/2021/10/4/22709850/facebook-instagram-whatsapp-down-outage

16. Kentik. (2021, October 5). *Facebook's historic outage explained*. Kentik Blog. https://www.kentik.com/blog/facebooks-historic-outage-explained/

17. Lomas, N. (2022, November 28). *Meta hit with 265 million euro fine for Facebook data scraping violations*. TechCrunch. https://techcrunch.com/2022/11/28/meta-dpc-scraping-fine/

18. Meta Platforms, Inc. (2021, October 5). *More details about the October 4 outage*. Meta Engineering Blog. https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/

19. Meta Platforms, Inc. (2021, October 4). *Update about the October 4th outage*. Meta Engineering Blog. https://engineering.fb.com/2021/10/04/networking-traffic/outage/

20. Meta Platforms, Inc. (2022). *Conveyor: One-tool-fits-all continuous software deployment at Meta*. Meta Engineering Blog. https://engineering.fb.com/2023/06/08/developer-tools/conveyor-meta-efficient-reliable-continuous-deployment-system/

21.Meta Platforms, Inc. (2023). *How Meta builds and manages its software development process*. Meta Engineering Blog. https://engineering.fb.com/2023/02/15/security/meta-bug-bounty-2022/

22. Meta Platforms, Inc. (2024). *Meta bug bounty program*. Meta Security. https://www.facebook.com/whitehat

23. Newman, L. H. (2024, January 30). *Meta patches Facebook account takeover vulnerability*. Security Week. https://www.securityweek.com/meta-patches-facebook-account-takeover-vulnerability/

24. NordVPN Team. (2021, April 6). *Facebook data breach: What happened and what to do now*. NordVPN Blog. https://nordvpn.com/blog/facebook-data-breach/

25. NPR. (2021, April 9). *After data breach exposes 530 million, Facebook says it will not notify users*. NPR. https://www.npr.org/2021/04/09/986005820/after-data-breach-exposes-530-million-facebook-says-it-will-not-notify-users

26. Ortutay, B. (2022, December 23). *Judge approves $725 million Facebook settlement over Cambridge Analytica scandal*. Associated Press. https://apnews.com/article/cambridge-analytica-facebook-meta-privacy-lawsuit-settlement-5b12d34e52ede13a19a5c30f4b55c0d5

27. Pressman, R. S., & Maxim, B. R. (2014). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill Education.

28. Rosenblatt, S. (2021, October 5). *Facebook outage: What happened and why it took so long to fix it*. CNET.

https://www.cnet.com/tech/services-and-software/facebook-outage-what-happened-and-why-it-took-so-long-to-fix-it/

29. Satariano, A. (2022, November 28). *Meta fined $275 million for failing to protect data of 533 million Facebook users*. The New York Times. https://www.nytimes.com/2022/11/28/business/meta-facebook-data-fine.html

30. Skelton, M., & Pais, M. (2019). *Team topologies: Organising business and technology teams for fast flow*. IT Revolution Press.

31. Somerville, I. (2016). *Software engineering* (10th ed.). Pearson Education Limited.

32. Tang, C. (2023, July 14). *Continuous software deployment — Part 1 of stories behind Meta/Facebook's systems research papers*. LinkedIn. https://www.linkedin.com/pulse/stories-behind-metafacebooks-systems-research-papers-part-cq-tang

33. The Guardian. (2018, March 21). *Facebook scandal: I am deeply sorry, says Zuckerberg*. The Guardian. https://www.theguardian.com/technology/2018/mar/21/mark-zuckerberg-response-cambridge-analytica-facebook

34. Whittaker, Z. (2021, April 7). *Facebook will not tell its 530 million users whose data was exposed*. TechCrunch. https://techcrunch.com/2021/04/07/facebook-data-breach-500-million-users/

35. Zuckerberg, M. (2018, March 21). *I want to share an update on the Cambridge Analytica situation*. Facebook. https://www.facebook.com/zuck/posts/10104712037900071

8. APPENDIX

SDT4202_GroupNo_2 .pdf

ORIGINALITY REPORT

3%	1%	2%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Barnsley College, South Yorkshire Student Paper	1%
2	Submitted to Islington College,Nepal Student Paper	<1%
3	Submitted to University of Exeter Student Paper	<1%
4	B.H. Far. "Software agents: quality, complexity and uncertainty issues", Proceedings First IEEE International Conference on Cognitive Informatics, 2002 Publication	<1%
5	www.productcool.com Internet Source	<1%
6	Submitted to INTI Universal Holdings SDM BHD Student Paper	<1%
7	cyber-gateway.net Internet Source	<1%
8	Che-Wei Chang, Cheng-Ru Wu, Hung-Lung Lin. "A simplified measurement scheme for software quality", Journal of Information and Optimization Sciences, 2006 Publication	<1%
9	Raymond Owen, Adam Bryant, Leigh Finch, Daniel Franklin, Mahrokh Abdollahi, Mehran Abolhasan. "Failures and Resilience in the IP	<1%

	Era: Navigating the Fragility of Modern Telecommunications Networks—The Sovereign Functions", IEEE Access, 2025 Publication	
10	Submitted to University of Lancaster Student Paper	<1%
11	Submitted to Wawasan Open University Student Paper	<1%
12	Nunes, Joana Filipa Loureiro. "The Unclarified Concept of Risk and Risk Assessments in Data Protection Impact Assessments", Universidade NOVA de Lisboa (Portugal) Publication	<1%
13	Alan Tang. "Privacy in Practice - Establish and Operationalize a Holistic Data Privacy Program", CRC Press, 2023 Publication	<1%
14	gucdoe.in Internet Source	<1%
15	Dr. Jason Edwards. "Mastering Cybersecurity", Springer Science and Business Media LLC, 2024 Publication	<1%
16	Ismail Suardi Wekke, Muhamad Ali Kasri, Lutfi Ali Muharom, Rizqi Claudia Wardani. "Evaluation of The New Student Candidates admission Information System Using ISO/IEC 25010 Model", Open Science Framework, 2021 Publication	<1%
17	Julius Murumba, Jackson Kipchirchir Machii. "chapter 13 ERP Software Inspections and Audits", IGI Global, 2020 Publication	<1%

18	Lee, Hyunsung Hazel. "How Do Ad-Blockers Affect Various Stakeholders? Perspectives from Consumers, Marketers, and Legal Professionals", Indiana University, 2023 Publication	<1%
19	Yaru Hao, Baogui Xin, Wei Peng. "Strategic resilience investment for a metaverse supply chain", RAIRO - Operations Research, 2025 Publication	<1%

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		